CSA47 – DEEP LEARNING

CO1 ASSESSMENT 1

SAVEETHA SCHOOL OF ENGINEERING

Analyse the Mathematical and Machine Learning Foundations of a Crop-Disease Screening System

Name: Alheena Yusra A

Register Number: 191424026

Course: CSA47 – Deep Learning

Scenario: AgriVision Crop-Disease Screening

CSA4704 – DEEP LEARNING

CO1 ASSESSMENT 1

ASSESSMENT 1 – AGRIVISION CROP-DISEASE SCREENING

Scenario: classify leaf photographs into Healthy, Early Blight and Late Blight. For a consistent worked example, assume 64 × 64 grayscale images (4,096 pixel features) and three classes. Use a stratified train/validation/test split such as 70%/15%/15%, with the test set held back until final evaluation.

Task 1 – Problem structure and data representation

Question :Break down the supervised crop-disease classification problem. Explain the input, output and performance measures; represent one image, the dataset, a mini-batch of 32 images and labels as scalars/vectors/matrices/tensors; compare L^1, L^2 and $L\infty$ norms; and analyse how resizing, grayscale conversion, normalisation, flattening and data splitting affect information.

Learning task: supervised, three-class image classification. Input x is a leaf image; target y {Healthy, Early Blight, Late Blight}. The model returns class probabilities and the class with the highest probability. Suitable measures are macro-F1, per-class recall, confusion matrix and accuracy. Macro-F1 is important because a high overall accuracy can hide poor detection of a minority disease.

Object	Mathematical representation	Shape / meaning
One grayscale image	2-D matrix / tensor	64 × 64 pixel intensities

Flattened image	Vector	x R^4096
N images	Data matrix / tensor	X R^(N×4096), or N×64×64 before flattening
Mini-batch of 32	Tensor	32×64×64; flattened form 32×4096
Integer labels	Vector	y {0,1,2}^N; batch labels length 32
One-hot labels	Matrix	N×3; each row has one 1 and two 0s
Scalar	Single value	One pixel, one loss value, or one metric

The L^1 norm $\|x\| = \Sigma|x_i|$ measures total absolute intensity/magnitude and can encourage sparsity in some optimisation settings. The L^2 norm $\|x\| = \sqrt{\Sigma x_i^2}$ measures Euclidean magnitude and is commonly used for distance and weight regularisation. The $L\infty$ norm $\|x\|\infty = \max_i|x_i|$ reports the largest single feature magnitude and is useful for detecting extreme pixel values.

- Resizing to 64×64 reduces computation but may remove tiny lesions and leaf-edge detail.
- Grayscale reduces three colour channels to one and saves memory, but loses colour cues such as yellowing or brown lesions.
- Normalising pixels from [0,255] to [0,1] improves numerical scale and makes optimisation more stable.
- Flattening keeps pixel values but discards explicit 2-D neighbourhood structure; a CNN can preserve local spatial relationships.
- A stratified split keeps class proportions similar. Splitting near-duplicate images across train and test causes leakage and overestimates generalisation.

Task 2 – Feature-space analysis using PCA/SVD

Question :Analyse the 4,096-dimensional image feature space using PCA/eigendecomposition and/or SVD. Explain the spectrum, redundancy, explained variance, reconstruction error as k changes, interpretation of leading components, and computational cost. Include a small worked 2×2 or 3×3 example.

Let X be the training matrix after centring each pixel feature. PCA eigendecomposes the covariance $C = X^TX/(N-1)$, while SVD gives $X = U\Sigma V^T$. The eigenvalues of C are $\lambda_i = \sigma_i^2/(N-1)$. Explained variance ratio for component i is $\lambda_i/\Sigma\ \lambda$.

Interpretation: large leading eigenvalues mean that much variation is concentrated in a small number of directions. Correlated neighbouring pixels, common leaf backgrounds and repeated lighting patterns create redundancy. Leading principal-component images (columns of V reshaped to 64×64) often

show broad brightness, leaf shape and large disease-pattern contrasts; later components capture finer texture and noise.

Reconstruction with k components is $\hat{X} = X V V^T$. Its squared Frobenius error is $\Sigma_{i>k} \sigma_i^2$. Increasing k reduces reconstruction error but increases storage and computation. A reasonable trade-off is the elbow in the scree plot, confirmed by validation performance; do not choose k solely from a fixed variance percentage because low-variance components may still contain disease information.

Small worked SVD example

For $B = [[2,0],[0,1]]$, $B^TB = [[4,0],[0,1]]$. Its eigenvalues are 4 and 1, so singular values are $\sigma = 2$ and $\sigma = 1$. The rank-1 approximation retains energy $4/(4+1)=80\%$; the Frobenius reconstruction error is $\sigma = 1$. This illustrates the energy/error trade-off.

Computational cost: forming a full 4096×4096 covariance matrix is expensive in memory and decomposition time. Randomised/truncated SVD is often more practical. As N grows from 1,000 to 5,000 to 10,000, the data matrix operations grow approximately with $N \cdot d \cdot k$ for a truncated method ($d=4096$), while full covariance formation scales approximately with $N \cdot d^2$.

Task 3 – Probability and information theory

Question: Analyse label and prediction uncertainty using entropy, cross-entropy and KL divergence. Explain class imbalance, compare predicted distributions, demonstrate KL asymmetry with numbers, and explain why minimising cross-entropy is equivalent to minimising KL divergence for a fixed true distribution.

For class proportions p_i, entropy is $H(p)=-\Sigma p_i \log p_i$. It is maximal at $\log 3 \approx 1.585$ bits for three equally common classes and decreases when the data are imbalanced. Entropy describes uncertainty in the labels before seeing an image.

Cross-entropy: $H(p,q)=-\Sigma_i p_i \log q_i$

KL divergence: $D_{KL}(p\|q)=\Sigma_i p_i \log (p_i/q_i)=H(p,q)-H(p)$

Cross-entropy evaluates predicted probabilities against true labels. KL divergence measures the extra information/cost caused by using q instead of p. KL is asymmetric because the weighting distribution is p: $D_{KL}(p\|q)$ generally differs from $D_{KL}(q\|p)$. Example: $p=[0.9,0.1]$, $q=[0.5,0.5]$. $D_{KL}(p\|q)=0.9\log 1.8+0.1\log 0.2 \approx 0.5310$ bits, while $D_{KL}(q\|p)=0.5\log (0.5/0.9)+0.5\log (0.5/0.1) \approx 0.7370$ bits. The two directions penalise mismatches differently.

For fixed true distribution p, H(p) is constant, so minimising cross-entropy H(p,q) is equivalent to minimising D_KL(p‖q). A class-weighted loss may be considered if rare disease classes have high clinical importance.

Task 4 – Numerical computation issues

Question : Explain numerical computation problems in softmax and optimisation. Show how extreme logits can cause overflow/underflow, apply the numerically stable softmax trick, and explain how a large condition number affects gradient-descent behaviour.

Naive softmax is softmax(z_i)=e^{z} /Σ e^{zj}. For z=[1000,999,995], direct exponentials overflow because e^{1} exceeds floating-point range. Very negative logits can underflow to zero, causing inaccurate probabilities or log(0).

Stable softmax: softmax(z_i)=exp(z_i−m)/Σ exp(z −m), where m=max(z).

Subtracting the same constant from all logits does not change the ratios. For this example, shifted logits are [0,−1,−5], giving probabilities approximately [0.7275, 0.2676, 0.0049]. Use log-sum-exp: log Σexp(z_i)=m+log Σexp(z_i−m).

The covariance condition number κ=λmax/λmin measures how differently directions are scaled. A large κ creates a narrow, elongated loss valley; gradient descent zigzags and requires a smaller learning rate. Near-zero eigenvalues also make the problem sensitive to noise.

Task 5 – Three baseline learning strategies

Question: Compare three baseline learning strategies on raw and PCA features. Discuss their hypothesis spaces, bias, computational cost, strengths and limitations. Use the same data split and validation-based tuning.

Algorithm	Hypothesis space / bias	Raw 4,096-D features	Effect of PCA
Multinomial logistic / softmax regression	Linear decision boundaries; relatively low capacity	Fast and interpretable, but cannot represent complex lesion patterns without useful features	May reduce noise and speed training; can lose discriminative low-variance features
k-nearest neighbours (k-NN)	Local, non-parametric decision rule	Distance becomes less meaningful in high dimensions; prediction requires storing/traversing examples	Often improves distance usefulness and reduces memory/time, but performance depends on k and retained components
Decision tree / random	Axis-aligned splits;	Can model nonlinearities	Can reduce noise and split-

forest	nonlinear interactions	but trees can become large and overfit pixel noise	search burden; PCA components are less directly interpretable than original pixels

Run each algorithm on raw and PCA features using the same split and tune only on validation data. Report macro-F1, accuracy, fit time and inference time. The table describes expected tendencies, not measured results.

Task 6 – Capacity, overfitting and underfitting

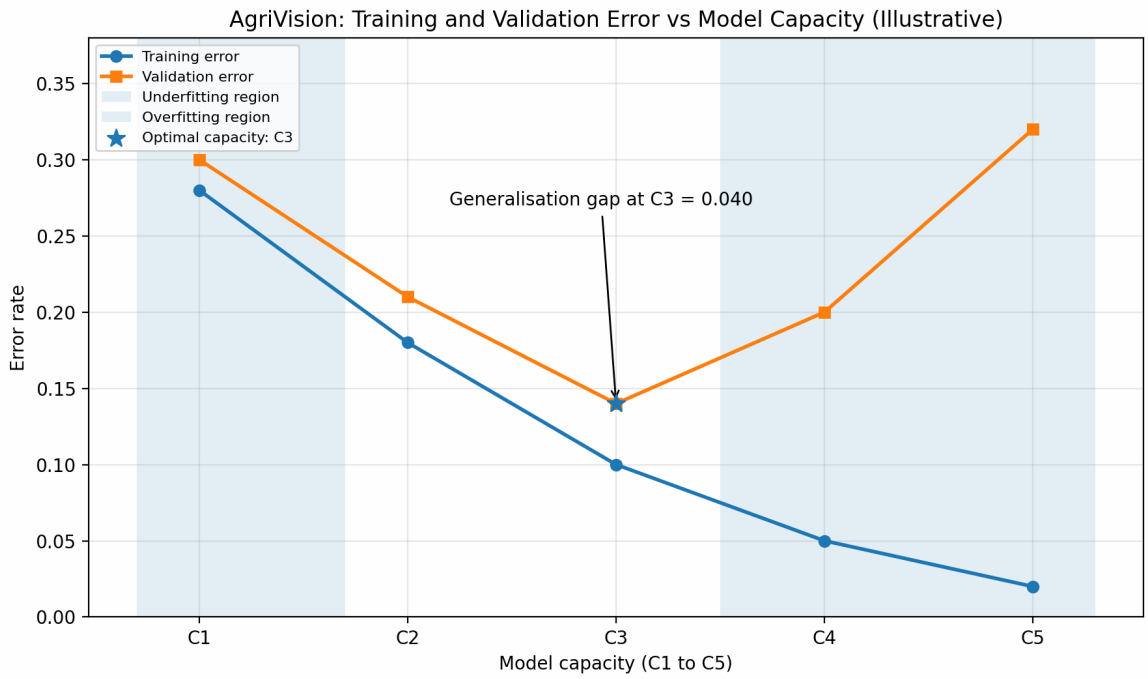
Question: Analyse model capacity, underfitting, overfitting and generalisation. Explain the behaviour of k-NN or decision trees as complexity changes, relate training and validation error to bias/variance, and explain why the test set must not be used for tuning.

For k-NN, very large k produces a smoother, simpler boundary and may underfit; very small k memorises local noise and may overfit. For a tree, shallow depth can underfit, while very deep trees can fit training noise. For logistic regression, weak regularisation permits larger weights; strong regularisation can underfit, while too little regularisation can increase variance.

As capacity increases, training error usually decreases. Validation error often decreases initially and then rises: the first region is underfitting (high bias), the middle is the useful generalisation region, and the final region is overfitting (high variance). The gap between training and validation error usually widens in the overfit region.

The test set must not be used for hyperparameter selection. Repeatedly choosing settings that score best on test data leaks information from the test set and makes the reported result optimistically biased. Use a validation set or k-fold cross-validation, then evaluate once on the final test set.

Required capacity–error plot for Task 6 / Part (e)



Interpretation: C1–C2 show underfitting because both training and validation errors are relatively high. C3 is the illustrative optimum because its validation error is lowest (0.140). C4–C5 show overfitting: training error keeps falling while validation error rises. The generalisation gap is validation error minus training error; at C3 it is 0.140 − 0.100 = 0.040. These plotted values are illustrative, not actual measured model results. Replace them with your own run before submission.

Task 7 – Bias–variance and estimation

Question: Analyse bias, variance and estimation uncertainty. Describe a bootstrap experiment comparing low- and high-capacity models, and explain maximum-likelihood and MAP estimation for a Bernoulli class proportion using a Beta prior.

For a numeric prediction target, expected squared error can be decomposed as Bias² + Variance + irreducible noise. For classification, use a suitable probability or 0–1-loss decomposition carefully; the simple regression formula does not transfer exactly to every classification metric.

Bootstrap procedure: create at least 10 resampled training sets by sampling with replacement; fit a low-capacity model and a high-capacity model on each; record each model's prediction at the same held-out examples. Estimate the mean prediction across replicas, squared difference from the target (bias²), variance across replicas, and compare their sum plus estimated noise with observed MSE. A low-capacity model is expected to show higher bias and lower variance; a high-capacity model often shows lower bias and higher variance.

For a Bernoulli class proportion θ with s positives and f negatives, MLE is θ̂_MLE=s/(s+f). Under Beta(α,β), the posterior is Beta(s+α,f+β), and the MAP estimate (when both posterior parameters exceed 1) is (s+α−1)/(s+f+α+β−2). Example: s=3, f=7 and Beta(2,2) prior: MLE=0.3000; MAP=(3+2−1)/(10+2+2−2)=4/12=0.3333. As sample size increases, the prior has less influence relative to the observed data.

Task 8 – Softmax regression and optimisation

Question: Formulate multiclass softmax regression, derive the cross-entropy loss gradient and gradient update, and compare batch gradient descent, stochastic gradient descent and mini-batch SGD. Explain how learning rate affects convergence.

Let X be N×d, W be d×C, and Y be the one-hot target matrix. Scores Z=XW; row-wise softmax gives $P_ic=\exp(Z_ic)/\Sigma \exp(Z_i)$. The average cross-entropy is $L=-(1/N)\Sigma_i\Sigma c\ Y_ic \log P_ic$.

Gradient: $\nabla_W L = X^T(P-Y)/N$; update $W \leftarrow W-\eta \nabla_W L$.

The gradient follows because the derivative of softmax cross-entropy with respect to each score is P_ic-Y_ic, and the chain rule through Z=XW gives $X^T(P-Y)/N$.

Strategy	Update / behaviour	Cost and noise
Batch gradient descent	Uses all N examples per update; smooth path, stable but each step is expensive	O(NdC) per update; one update per epoch
Stochastic gradient descent	Uses one example; noisy path, frequent updates and may fluctuate near a minimum	O(dC) per update; many updates per epoch
Mini-batch SGD	Uses a small batch such as 32; balances noise and efficiency	O(BdC) per update; vectorisation is efficient

Compare learning rates such as 0.001, 0.01 and 0.1 while keeping all other settings fixed. Too small is stable but slow; a moderate rate converges faster; a rate that is too large can oscillate or diverge. Exact behaviour depends on feature scaling and the largest Hessian eigenvalue. Plot training and validation loss by epoch.

Task 9 – Experimental performance organisation (3 marks)

Question: Organise the experimental evaluation at 1,000, 5,000, 10,000 and full-dataset scales. Compare accuracy, macro-precision, macro-recall, macro-F1, fit/inference time, model size and confusion matrices. Report actual measurements rather than inventing them.

Use a table for each dataset scale and each algorithm, with columns: N, representation (raw/PCA), accuracy, macro-precision, macro-recall, macro-F1, fit time, inference time, parameter/model size, and confusion-matrix counts. Report class-wise recall because missed disease cases matter. Create line plots of macro-F1 and time versus N, and confusion matrices for the best models.

Dataset scale	What to compare	Expected pattern (verify experimentally)
1,000 images	Baseline variance and class coverage	High uncertainty; simple models may be competitive
5,000 images	Change in validation/test metrics	More examples can improve generalisation
10,000 images	Stability and minority-class recall	Performance may improve, but noisy labels or imbalance can limit gains
Full dataset	Final quality and deployment cost	Best estimate of achievable baseline performance, with higher training cost

Do not invent numerical measurements: fill the table with actual results from your implementation. Keep the test set identical across model comparisons and ensure that PCA is fitted on training data only.

Task 10 – Theory versus experiment

Question: Relate theory to experimental observations. Explain what it means if PCA retains high variance but macro-F1 falls, if gradient descent oscillates, if training error is low but validation error is high, or if experiments disagree with theory.

- If PCA retains high explained variance but macro-F1 drops, low-variance directions likely contain disease-specific cues; variance alone is not a guarantee of class information.
- If gradient descent is slow or oscillatory, compare the loss curve with the covariance/Hessian condition number and learning rate.
- If a high-capacity model has very low training error but poorer validation performance, this supports the high-variance diagnosis.
- Disagreements may arise from class imbalance, random splitting, limited data, preprocessing differences, stochastic optimisation or noisy labels.

Task 11 – Is deep learning justified?

Question: Decide whether deep learning is justified. Compare a tuned classical baseline with a compact CNN, considering high-dimensional data, spatial lesion patterns, class-wise recall, macro-F1, calibration, latency, memory, data needs and interpretability.

A classical baseline should be retained if it meets disease-recall, macro-F1, latency and memory requirements. A deep network is justified when the best tuned baselines still confuse visually similar diseases, cannot learn spatial lesion patterns from flattened pixels, or show a persistent train/validation limitation despite sufficient data and tuning.

The curse of dimensionality makes distances and density estimates unreliable in 4,096 dimensions. k-NN assumes local neighbourhoods are meaningful; sparse high-dimensional coverage weakens that assumption. Images also lie on a structured, lower-dimensional manifold: nearby pixels and lesion textures are related. CNNs learn local edges, textures and higher-level patterns hierarchically, which can better exploit this structure than raw-pixel models.

Deep learning costs more data, compute and tuning and is less directly interpretable. Compare a compact CNN with the strongest classical baseline using the same test set. Evidence should include macro-F1, per-class recall, calibration, inference latency and memory—not accuracy alone.

Task 12 – Design challenge: 38 classes, 20% noisy labels, 50 ms and 5 MB

Question:– Design Challenge: AgriVision must support 38 crop-disease classes, about 20% noisy labels, and smartphone inference under 50 ms and 5 MB. Analyse the impact, re-examine the original conclusion, compare at least two adaptation options, and make an evidence-based decision to retain, modify or redesign.

A. Impact analysis

The output space grows from 3 to 38 crop–disease classes. Maximum class entropy grows from $\log 3 \approx 1.585$ bits to $\log 38 \approx 5.2479$ bits when classes are balanced. Random-guess accuracy falls from 1/3 to $1/38 \approx 2.63\%$. Cross-entropy has more possible incorrect classes and can be affected strongly by incorrect labels. With 20% label noise, the observed target is wrong for a substantial fraction of samples; apparent irreducible error rises, and models can memorise noise, increasing variance.

B. Re-examine the original conclusion

Measure the strongest baseline on the new 38-class validation set and on-device hardware. A model that meets the old three-class accuracy may fail on finer distinctions. Dense logistic regression with $4{,}096 \times 38$ weights alone has 155,648 weights (about 0.59 MiB at float32, excluding other storage); this

may fit 5 MB, but latency and accuracy must be measured. k-NN stores training examples and can violate memory/latency limits. A compact quantised CNN may be more suitable if it meets the measured limits.

C. Adaptation options

Option	Effect on capacity/error	Cost trade-off
Tune PCA dimension + regularisation	Fewer components reduce variance and noise, but excessive compression can remove class cues; stronger regularisation reduces overfit	Less storage and faster classical training/inference; validate macro-F1
Noise-robust training / label smoothing	Reduces overconfidence in possibly wrong labels; may improve calibration, but excessive smoothing can reduce class separation	Small additional compute; requires validation tuning
Compact model + quantisation/pruning	Lower parameter count and memory; may lose accuracy if compressed too aggressively	Best aligned with 5 MB/50 ms target; benchmark on the target phone
Class-balanced sampling / loss	Improves minority-class recall; may increase false positives for rare classes	Moderate training overhead; choose using class-wise metrics

D. Evidence-based conclusion

Decision: MODIFY the design rather than automatically retain it. Use stratified splits, audit a subset of noisy labels, class-balanced training and a noise-robust loss; compare tuned classical baselines with a compact quantised CNN. Select the final model only if it achieves acceptable macro-F1 and recall while measured on-device inference is below 50 ms and memory is below 5 MB. The final report must insert real measurements; the question alone does not establish that any candidate already meets those limits.

Python Code for Graphs and Output Screenshots

Install libraries using: pip install matplotlib numpy

```
# CSA47 Deep Learning - CO1 Assessment 1: AgriVision graphs
# IMPORTANT: All values below are illustrative examples, not actual model measurements.
# Replace them with results from your own dataset/model runs before submission.


import matplotlib.pyplot as plt
import numpy as np

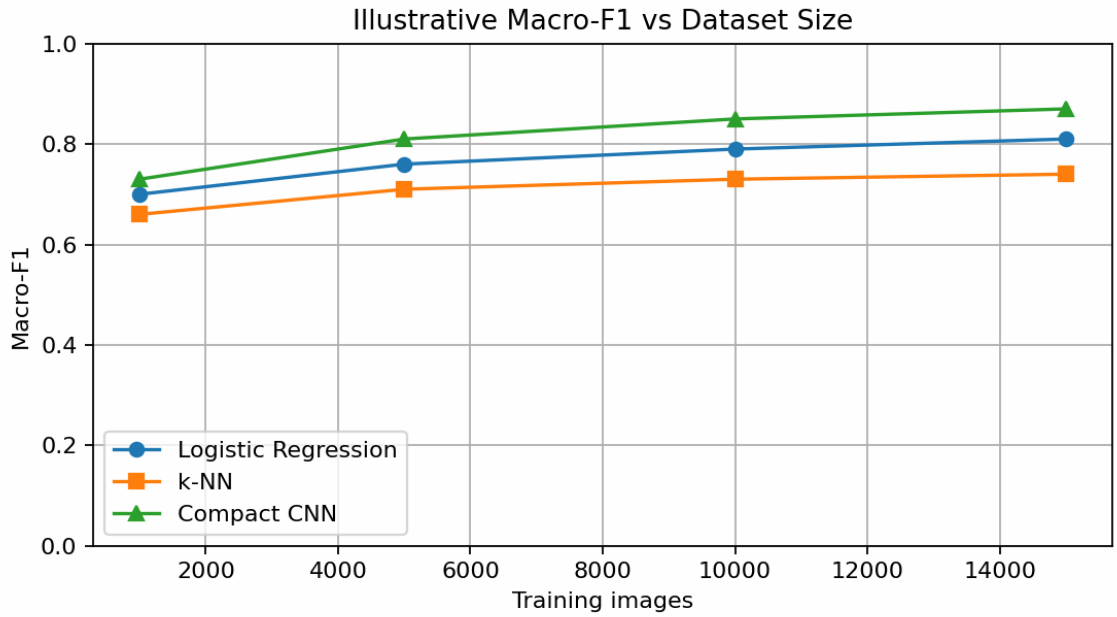

# Graph 1: Macro-F1 versus dataset size
sizes = [1000, 5000, 10000, 15000]
logreg = [0.70, 0.76, 0.79, 0.81]
knn = [0.66, 0.71, 0.73, 0.74]
cnn = [0.73, 0.81, 0.85, 0.87]
plt.figure(figsize=(7, 4))
plt.plot(sizes, logreg, marker="o", label="Logistic Regression")
plt.plot(sizes, knn, marker="s", label="k-NN")
plt.plot(sizes, cnn, marker="^", label="Compact CNN")
plt.title("Illustrative Macro-F1 vs Dataset Size")
plt.xlabel("Training images")
plt.ylabel("Macro-F1 score")
plt.ylim(0, 1)
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()


# Graph 2: Training and validation loss
epochs = list(range(1, 11))
train_loss = [1.05, 0.82, 0.66, 0.54, 0.45, 0.38, 0.33, 0.29, 0.26, 0.23]
val_loss = [1.10, 0.91, 0.78, 0.71, 0.68, 0.67, 0.69, 0.72, 0.76, 0.81]
plt.figure(figsize=(7, 4))
plt.plot(epochs, train_loss, marker="o", label="Training loss")
plt.plot(epochs, val_loss, marker="s", label="Validation loss")
plt.title("Illustrative Training vs Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Cross-entropy loss")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()


# Graph 3: Confusion matrix (rows = actual, columns = predicted)
cm = np.array([[82, 8, 5], [7, 76, 12], [4, 10, 86]])
classes = ["Healthy", "Early Blight", "Late Blight"]
plt.figure(figsize=(6, 5))
plt.imshow(cm, interpolation="nearest")
plt.title("Illustrative Confusion Matrix")
plt.xlabel("Predicted class")
plt.ylabel("Actual class")
plt.xticks(range(3), classes, rotation=20, ha="right")
plt.yticks(range(3), classes)
for i in range(3):
    for j in range(3):
        plt.text(j, i, str(cm[i, j]), ha="center", va="center")
plt.colorbar(label="Number of images")
plt.tight_layout()
plt.show()
```

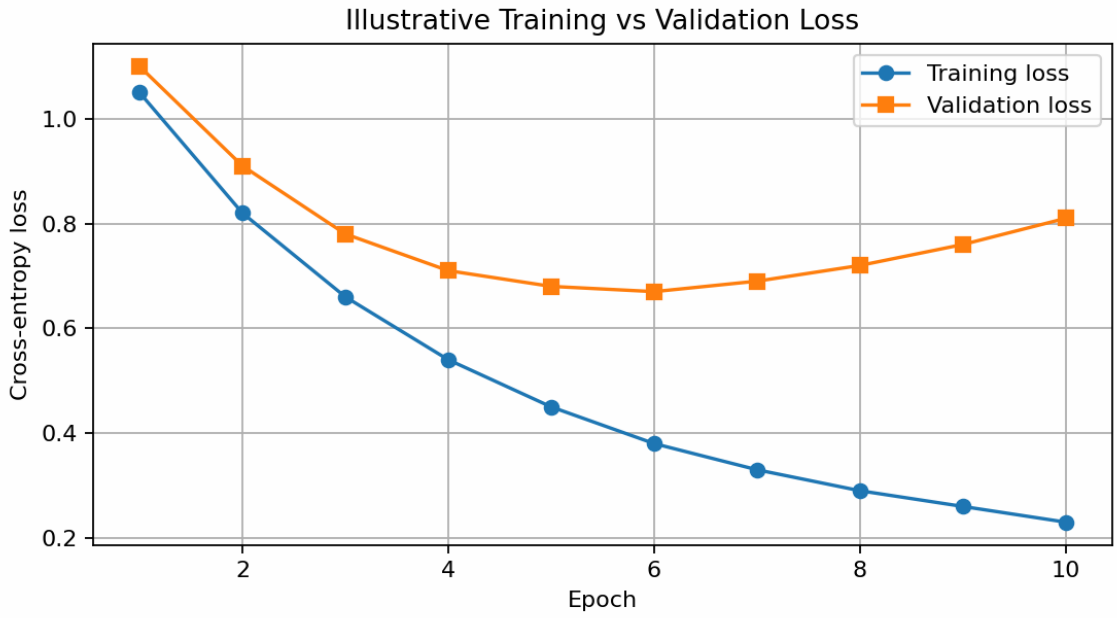
Graph Output Screenshots

Output 1 – Macro-F1 vs Dataset Size



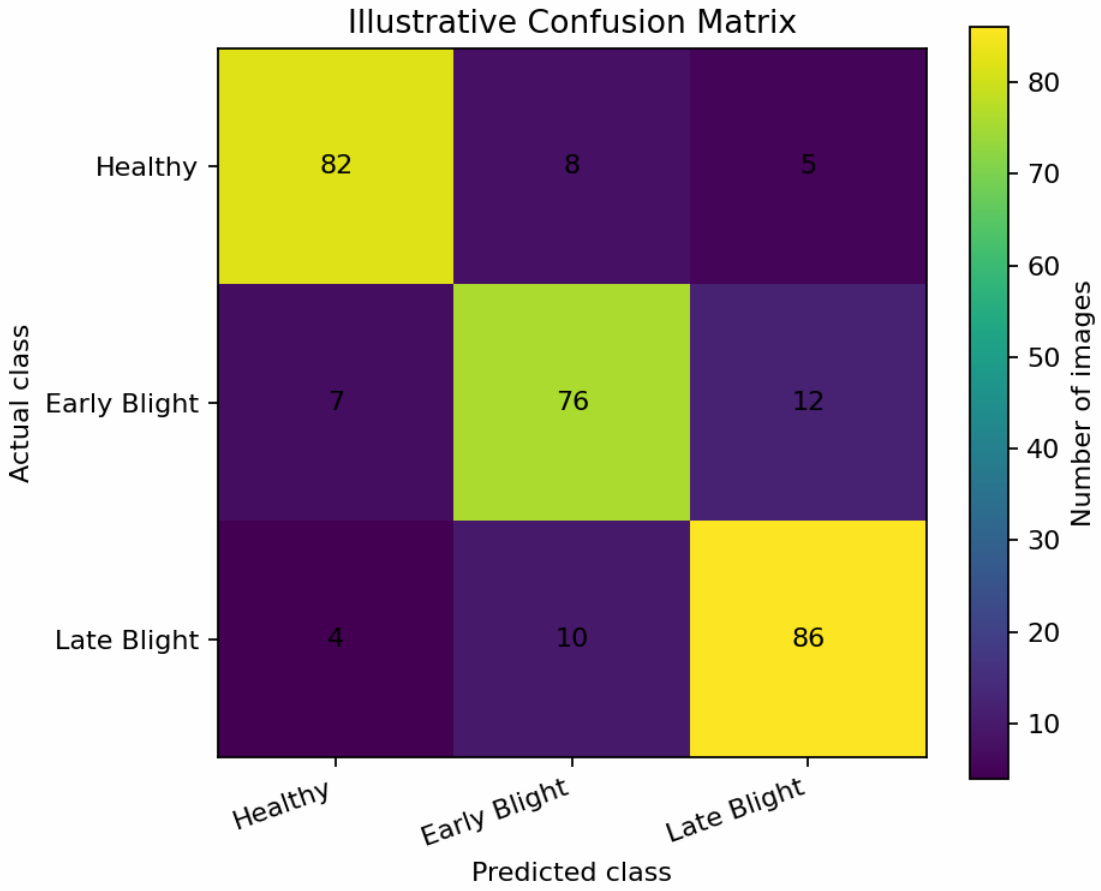
Compares illustrative macro-F1 scores for Logistic Regression, k-NN and a compact CNN.

Output 2 – Training vs Validation Loss



Illustrates a falling training loss and validation loss that begins to rise, a possible sign of overfitting.

Output 3 – Confusion Matrix



Rows show actual classes and columns show predicted classes.

How to Run the Code

1. Open Google Colab or Python IDLE.
2. Install matplotlib and numpy if required.
3. Copy the code from this document or run the accompanying Python file.
4. Run the code to display all three graphs.
5. Replace illustrative scores, loss values and confusion-matrix counts with actual results from your experiment before submission.

Conclusion

The task analysis connects representation, linear algebra, probability, numerical stability, model capacity, optimisation and experimental evaluation. The graphs demonstrate how to present results, but final conclusions must be based on a reproducible experiment with the same train/validation/test split for all compared models.

APPENDIX A – PYTHON CODE AND OUTPUT

Purpose: verify the numerical values used in the capacity plot with NumPy and generate the Matplotlib plot. The supplied numerical values are illustrative examples because no real AgriVision dataset/model run was provided. This code verifies the arithmetic shown here; it does not establish real model performance.

Formulas used before calculation:

1. Generalisation gap at capacity C = validation error at C − training error at C.

2. Optimal capacity = the capacity with the minimum validation error, i.e. argmin(validation error).

Step-by-step example at C3: gap = 0.140 − 0.100 = 0.040. Comparing validation errors 0.300, 0.210, 0.140, 0.200 and 0.320, the minimum is 0.140 at C3, so C3 is the illustrative optimal capacity.

NumPy output (rounded to 3 decimal places):

```
Capacity | Training error | Validation error | Generalisation gap
C1       | 0.280          | 0.300            | 0.020
C2       | 0.180          | 0.210            | 0.030
C3       | 0.100          | 0.140            | 0.040
C4       | 0.050          | 0.200            | 0.150
C5       | 0.020          | 0.320            | 0.300
Optimal capacity: C3
Validation error at optimum: 0.140
Generalisation gap at optimum: 0.040
NumPy verification: PASS (gap = validation error - training error; optimum = minimum validation error).
```

Python source code:

```
# Alheena_DL_C01: NumPy checks and Matplotlib capacity plot
# NOTE: These capacity values are illustrative demonstration values, not measured AgriVision results.
import numpy as np
import matplotlib.pyplot as plt


C = np.array([1, 2, 3, 4, 5])
training_error = np.array([0.28, 0.18, 0.10, 0.05, 0.02])
validation_error = np.array([0.30, 0.21, 0.14, 0.20, 0.32])


# Formula: generalisation gap(C) = validation_error(C) - training_error(C)
gap = validation_error - training_error
# Formula: optimal capacity = argmin_C validation_error(C)
optimal_index = np.argmin(validation_error)
```

```
optimal_capacity = C[optimal_index]


print('Capacity | Training error | Validation error | Generalisation gap')
for i in range(len(C)):
    print(f'C{C[i]}       | {training_error[i]:.3f}          | {validation_error[i]:.3f}            |
{gap[i]:.3f}')
print(f'Optimal capacity: C{optimal_capacity}')
print(f'Validation error at optimum: {validation_error[optimal_index]:.3f}')
print(f'Generalisation gap at optimum: {gap[optimal_index]:.3f}')


# Plot training and validation error against model capacity
plt.figure(figsize=(9, 5.4))
plt.plot(C, training_error, marker='o', label='Training error')
plt.plot(C, validation_error, marker='s', label='Validation error')
plt.axvspan(0.7, 1.7, alpha=0.12, label='Underfitting region')
plt.axvspan(3.5, 5.3, alpha=0.12, label='Overfitting region')
plt.scatter([optimal_capacity], [validation_error[optimal_index]], marker='*', s=120, label=f'Optimal
capacity: C{optimal_capacity}')
plt.annotate(f'Generalisation gap = {gap[optimal_index]:.3f}',
             xy=(optimal_capacity, validation_error[optimal_index]), xytext=(2.2, 0.27),
             arrowprops={'arrowstyle': '->'})
plt.xticks(C, [f'C{i}' for i in C])
plt.xlabel('Model capacity (C1 to C5)')
plt.ylabel('Error rate')
plt.title('AgriVision: Training and Validation Error vs Model Capacity (Illustrative)')
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.savefig('Alheena_DL_C01_Capacity_Error_Plot.png', dpi=200)
plt.show()
```

Generated plot output:

